

The Complete Docker Cheat Sheet

Architecture, components, and every command you need —

1. What is Docker? (The Easy Explanation)

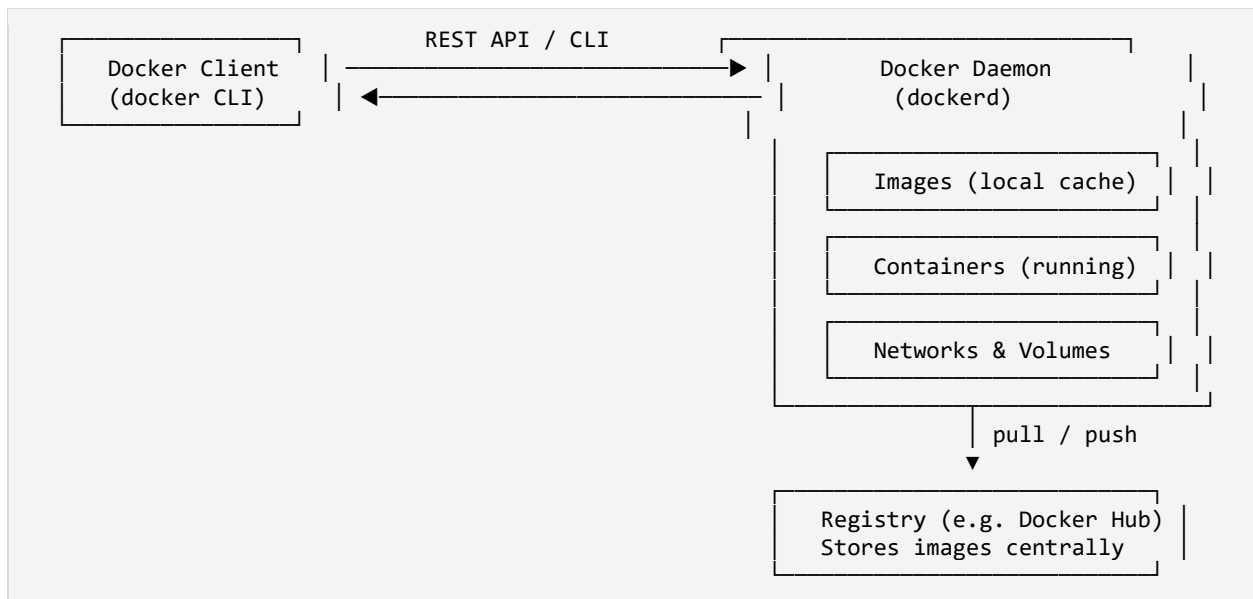
Docker is a tool that packages an application with everything it needs — code, runtime, system libraries, settings — into one unit called a container. That container behaves exactly the same on your laptop, your teammate's laptop, or a production server, because it carries its environment with it instead of depending on what's already installed on the machine.

Easy analogy: A container is like a sealed lunchbox. Whatever is packed inside travels with it and tastes the same wherever you open it. Without Docker, you're relying on the kitchen at your destination already having the right ingredients — which often causes the classic “it works on my machine” problem.

2. Docker Architecture — How It All Fits Together

Docker uses a client-server architecture. You type commands into a client, and a background service (the daemon) does the actual work of building, running, and managing containers.

Architecture diagram (simplified)



Walking through the architecture, piece by piece

- **Docker Client** — what you type into your terminal (e.g. `docker run`, `docker build`). The client doesn't do the actual work itself — it just sends your commands to the daemon over a REST API.
- **Docker Daemon (dockerd)** — a background process (a service) running on your machine that does the real work: building images, starting/stopping containers, managing networks and storage. This is the “engine” of Docker.
- **Local Image Cache** — the daemon keeps a local copy of any images you've pulled or built, so it doesn't need to re-download them every time.
- **Containers** — when you run a container, the daemon creates it from an image and manages its lifecycle (start, stop, pause, remove).
- **Registry** — a remote server that stores images so they can be shared. Docker Hub is the default public registry; companies often run private registries too.
- **Pull / Push** — when an image isn't available locally, the daemon contacts the registry to fetch it (`docker pull`). You can also upload your own images there (`docker push`).

Easy way to remember it: You (the client) place an order. The daemon is the kitchen that actually cooks it, using ingredients (images) either already in the pantry (local cache) or fetched from a supplier (registry).

3. All the Core Components, Explained Simply

3.1 Image

A read-only template that contains an application and everything it needs to run. Think of it as a frozen recipe — it doesn't change once built, and you can use it to create as many containers as you want.

- Built in layers — each instruction in a Dockerfile (`FROM`, `COPY`, `RUN`, etc.) creates a new layer stacked on top of the previous one
- Layers are cached and reused, which is why rebuilding an image after a small code change is usually fast

3.2 Container

A running instance of an image. If the image is the recipe, the container is the actual dish made from it — it has a live process, can be started/stopped, and can change while it runs (though those changes disappear when the container is removed, unless you're using a volume).

3.3 Dockerfile

A plain text file with step-by-step build instructions. Docker reads it top to bottom and creates one image layer per instruction.

Instruction	What it does
FROM	Sets the base image to start from
WORKDIR	Sets the working directory inside the image
COPY / ADD	Copies files from your machine into the image
RUN	Executes a command during the build (e.g. installing packages)
ENV	Sets an environment variable inside the image
EXPOSE	Documents which port the app listens on (informational only)
ENTRYPOINT / CMD	Defines the command that runs when a container starts

3.4 Docker Compose

A tool for defining and running multiple containers together using one YAML file (docker-compose.yml), instead of starting each one manually. Useful when your app depends on other services like a database or a message broker.

3.5 Registry

A storage and distribution system for images. Docker Hub is the default public registry; private registries (e.g. AWS ECR, Google Artifact Registry, Harbor) are common in companies for proprietary images.

3.6 Volume

Containers are disposable by design — when you remove one, its filesystem changes are lost. A volume is storage that lives outside the container's own filesystem, so data survives even if the container is deleted or recreated. Used for things like databases that need to persist data.

3.7 Network

Docker creates isolated virtual networks so containers can talk to each other (and optionally to the outside world). By default, containers on the same custom network can reach each other using their container/service name as a hostname.

3.8 Docker Engine

The overall name for the underlying technology that makes all of this work — it includes the daemon, the API, and the CLI together.

3.9 Component Comparison — At a Glance

Component	Analogy	Lifespan
Image	Recipe / blueprint	Static — doesn't run, just stored
Container	Cooked dish from the recipe	Lives while running; removable anytime
Volume	A fridge outside the kitchen	Persists even if the container is thrown away
Network	The delivery roads between kitchens	Exists as long as you keep it / Compose is up
Registry	The supplier's warehouse	Long-term storage, shared across machines

4. Command Cheat Sheet

4.1 Images

Command	What it does
<code>docker pull <image></code>	Downloads an image from a registry
<code>docker build -t <name> .</code>	Builds an image from a Dockerfile in the current folder
<code>docker images</code>	Lists all images stored locally
<code>docker rmi <image></code>	Deletes a local image
<code>docker tag <image> <new-name></code>	Creates an additional name/tag for an existing image
<code>docker push <image></code>	Uploads an image to a registry
<code>docker history <image></code>	Shows the layer-by-layer build history of an image

4.2 Containers

Command	What it does
<code>docker run <image></code>	Creates and starts a new container from an image
<code>docker run -d <image></code>	Runs the container in the background (detached)
<code>docker run -p 8080:80 <image></code>	Maps host port 8080 to container port 80
<code>docker run --name myapp <image></code>	Gives the container a friendly name
<code>docker run -e KEY=value <image></code>	Sets an environment variable inside the container

Command	What it does
<code>docker run -v myvol:/data <image></code>	Mounts a volume into the container
<code>docker ps</code>	Lists running containers
<code>docker ps -a</code>	Lists all containers, including stopped ones
<code>docker stop <container></code>	Gracefully stops a running container
<code>docker start <container></code>	Starts a previously stopped container
<code>docker restart <container></code>	Restarts a container
<code>docker rm <container></code>	Deletes a stopped container
<code>docker exec -it <container> bash</code>	Opens an interactive shell inside a running container
<code>docker logs -f <container></code>	Streams live logs from a container
<code>docker inspect <container></code>	Shows detailed configuration/metadata as JSON
<code>docker stats</code>	Shows live CPU/memory usage for running containers

4.3 Volumes

Command	What it does
<code>docker volume create myvol</code>	Creates a named volume
<code>docker volume ls</code>	Lists all volumes
<code>docker volume inspect myvol</code>	Shows details about a volume
<code>docker volume rm myvol</code>	Deletes a volume

4.4 Networks

Command	What it does
<code>docker network create mynet</code>	Creates a custom network
<code>docker network ls</code>	Lists all networks
<code>docker network inspect mynet</code>	Shows details about a network, including connected containers
<code>docker network connect mynet <container></code>	Attaches a running container to a network

4.5 Docker Compose

Command	What it does
<code>docker compose up -d</code>	Builds (if needed) and starts all services in the background
<code>docker compose up -d --build</code>	Forces a rebuild of images before starting
<code>docker compose ps</code>	Lists services defined in the Compose file and their status
<code>docker compose logs -f</code>	Streams logs from all services
<code>docker compose logs -f <service></code>	Streams logs from one specific service
<code>docker compose stop</code>	Stops services without removing them
<code>docker compose down</code>	Stops and removes containers, networks created by Compose
<code>docker compose down -v</code>	Same as above, but also removes named volumes

4.6 Cleanup Commands

Command	What it does
<code>docker system df</code>	Shows disk space used by images, containers, volumes
<code>docker container prune</code>	Removes all stopped containers
<code>docker image prune</code>	Removes unused (dangling) images
<code>docker volume prune</code>	Removes unused volumes
<code>docker system prune -a</code>	Removes all unused containers, images, and networks (use with care)

5. Dockerfile Quick Reference

A minimal but complete example showing the most commonly used instructions together:

```
FROM eclipse-temurin:17-jre-alpine # base image to start from
WORKDIR /app                      # working directory inside the image
COPY target/*.jar app.jar         # copy a local file into the image
ENV SPRING_PROFILES_ACTIVE=prod  # set an environment variable
EXPOSE 8080                       # document the port the app uses
ENTRYPOINT ["java", "-jar", "app.jar"] # command to run on container start
```

CMD vs ENTRYPOINT: ENTRYPOINT defines the fixed command that always runs; CMD defines default arguments that can be overridden when starting the container. Many simple images use CMD alone; combining both gives you a fixed executable with overridable defaults.

6. Docker Compose Quick Reference

```
version: "3.8"

services:
  app:
    build: .                # build from Dockerfile in current folder
    ports:
      - "8080:8080"        # host:container port mapping
    environment:
      - SPRING_PROFILES_ACTIVE=docker
    depends_on:
      - db                  # start db before app
    volumes:
      - app-data:/app/data  # mount a named volume

  db:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=secret

volumes:
  app-data:                 # declares the named volume used above
```

Key reminder: Inside Compose, containers reach each other using the service name as the hostname (e.g. db:5432), never localhost — localhost inside a container always refers to that container itself.

Quick Recap

Term	One-line meaning
Docker Client	What you type commands into
Docker Daemon	The background engine that does the actual work
Image	Read-only template/blueprint for a container
Container	A running instance of an image
Dockerfile	Instructions for building an image
Docker Compose	Run and coordinate multiple containers together

Term	One-line meaning
Volume	Storage that survives outside the container's life
Network	Lets containers talk to each other by name
Registry	Central place to store and share images (e.g. Docker Hub)